Model choice justification

Why each model does, or does not, fit this problem: theory, our data, and the evidence · IT3091 Machine Learning · Group 2026-DS-09

Owner: Bandara H. M. T. A. (IT24103675), modelling and evaluation · for all members to use when answering questions

How to read this: Section 1 lists the properties of our data that decide which models can work. Section 2 explains every model, used or considered, against those properties, with the validation result. Section 3 explains why the best models end up tied. Results are validation Macro-F1 on the 2023/24 season (2,802 matches); the test seasons were used only once, for the final model.

1. What our data looks like

Property	Evidence	What it means for the choice of model
P1 Medium-sized table	42,795 training matches × 38 features (37 numeric + league)	Tree ensembles work well on tables of this size; neural networks have little room to learn extra structure; kernel SVMs get slow (cost grows with rows²).
P2 Main signal is smooth and one-directional	XI strength difference has the largest effect (ε^2 0.145); +1 SD multiplies the home-win vs draw odds by 1.53	Even a linear model captures most of the signal, so extra flexibility adds little.
P3 Strongly overlapping features	XI vs top-3 strength r = 0.94; 14 of 26 features have VIF above 10; several exact identities	Breaks the independence assumption of Naive Bayes; distorts distance-based kNN; harmless for the predictions of trees and regularised models (but importance must then be read by feature group).
P4 Missing values	Up to 3.1% per FS3 feature in the training seasons (structural: season openers, unrated starters)	Needs train-only imputation, or a model that handles missing values natively (HistGradientBoosting, XGBoost).
P5 Three unbalanced, ordered classes	Home 45.0%, Draw 24.9%, Away 30.1% (all 51,201 matches); Draw is the "middle" outcome	Without class weighting a model almost never predicts a draw. Ordered models put Draw in a middle band, where it is rarely the most likely class.
P6 Large random component	The stronger XI wins only 65.5-70.8% of matches that are not drawn	No model can remove the randomness, so flexible models reach the same ceiling (about 0.49 Macro-F1).
P7 Patterns drift over time	Outcome shares change slightly by season (Cramér's V 0.034); home advantage declines	Validation must be chronological; simpler, stable models generalise better than over-tuned ones.
P8 The user needs probabilities	The output is P(Home), P(Draw), P(Away) for decision support	Probability quality (log loss) matters. Models that need an extra calibration step to give probabilities (SVM) or give unreliable ones (Naive Bayes, kNN) are less useful.

2. Model by model

Original six models: ML notebook Sections 11-16 (scikit-learn 1.8). Models marked §26 were checked after the final model was frozen, in notebook Section 26 (scikit-learn 1.9), and compared with the frozen Random Forest retrained in the same run (validation Macro-F1 0.495). The Poisson goal model (§27) was run with scikit-learn 1.8, where the Random Forest scores 0.492. A new model would replace Random Forest only if it beat it by more than 0.01 on validation. None did.

Model	How it works (theory and assumption)	Fit with our data (P1-P8)	Validation result	Verdict	If the evaluator asks "why not this model?"
Dummy (class prior)	Predicts the training class shares for every match. Assumes no feature carries information.	Reference only.	0.201 · log loss 1.077	Baseline	It is the minimum bar; every real model beats it, by 0.20 to 0.29 Macro-F1.
Logistic Regression	Softmax of a linear score per class; L2 regularised. Assumes log-odds change linearly with the features.	Fits P2 well (smooth signal). P3 inflates coefficient variance but not predictions. Needs class weights (P5).	0.485 (balanced) · Draw F1 0.267 · log loss 0.984. Unweighted: 0.403, Draw F1 0.000	Good, close to the best	A linear model is only about 0.01 below the best, which shows the signal is mostly linear in team-strength differences.

Model	How it works (theory and assumption)	Fit with our data (P1-P8)	Validation result	Verdict	If the evaluator asks "why not this model?"
Gaussian Naive Bayes	Bayes rule with features assumed independent given the class, each a normal distribution.	P3 violates the independence assumption by construction (XI, top-3 and unit strengths are all built from the same player ratings), so the same evidence is counted several times.	0.473 · Draw F1 0.236 · log loss 2.353 (FS4: 0.373, log loss 5.80)	Rejected	Its Macro-F1 looks fine, but its probabilities are far too confident (log loss 2.35 vs 0.98), because correlated features are counted as independent evidence.
k-Nearest Neighbours	Class shares among the k most similar training matches (Euclidean distance on scaled features). Assumes similar inputs have similar outcomes.	P3: every feature gets equal weight in the distance, so many overlapping strength columns dominate it. P6: noisy labels need a large k. P5: scikit-learn's kNN has no class weights.	0.437 (tuned, k = 25) · Draw F1 0.150 · log loss 1.096 (worse than the Dummy, 1.077)	Weakest of the original six	Distance treats all 38 features as equally important and the model has no class weights, so it blurs the main signal, rarely predicts draws and gives poor probabilities.
Decision Tree	One tree of yes/no splits chosen to reduce Gini impurity.	P2: approximates a smooth trend with steps. One tree has high variance (unstable splits).	0.478 (balanced) · Draw F1 0.337 · log loss 0.993. Unweighted: 0.400	Good but unstable	It is interpretable but a single tree varies a lot; averaging many trees (Random Forest) fixes that.
Random Forest (FINAL)	Average of 200 trees, each grown on a bootstrap sample with a random subset of features tried at every split. Bagging reduces variance.	Handles P3 (correlation), P4 (with imputation) and non-linearity; class weights for P5; gives probabilities (P8); trains in seconds.	0.492 (tuned, balanced) · Draw F1 0.334 · log loss 0.983. Test: 0.473	Selected	Joint-best Macro-F1; log loss within about 0.004 of the best class-weighted model; few settings, fast, stable, and explained through grouped feature importance.
Gradient Boosting	Shallow trees added one by one, each correcting the previous errors (reduces bias).	P2/P6: there is little systematic error left to correct, so boosting ties with bagging.	0.491 (balanced) · Draw F1 0.324 · log loss 0.980. Unweighted: 0.407	Equivalent alternative	Effectively tied with Random Forest (0.001 lower); boosting cannot find signal that is not in the features.
HistGradientBoosting (§26)	Boosting on binned features (LightGBM approach); learns where to send missing values at each split.	P4 handled natively; same theory and ceiling as boosting.	0.495 vs Random Forest 0.495 in the same run (+0.00005) · Draw F1 0.327 · log loss 0.981	Tie, not adopted	Equal to Random Forest; the 0.00005 gain is far below the 0.01 decision rule and the noise level, so the simpler model that was already selected stays.
XGBoost (§26)	Regularised gradient boosting with second-order updates; native missing values.	Same as boosting.	0.493 (−0.002 vs Random Forest) · Draw F1 0.322 · log loss 0.980	Tie, not adopted	Tested: no gain. The limit is the information before kickoff, not the boosting algorithm.
AdaBoost (script 05)	The original boosting method: after each round, wrongly predicted matches get more weight, so the next shallow tree concentrates on them.	P2/P6: little systematic error is left, and many wrong predictions are upsets no pre-match information can explain, so later rounds chase noise.	0.482 (−0.013 vs Random Forest) · Draw F1 0.320 · log loss 1.059	Below Random Forest	Tested: lower Macro-F1 and the weakest probabilities of the tree models; validation log loss rises with every extra round (1.032 after 10 rounds, 1.060 after 300).
MLP neural network (§26)	Layers of weighted sums with non-linear activations, trained by gradient descent; with enough hidden units it can approximate any smooth function.	P1/P2: a 38-feature table with a smooth signal leaves nothing for hidden layers to discover; needs scaling and tuning.	0.485 (balanced, 64-32 units) · Draw F1 0.305 · log loss 0.980. Unweighted: 0.414, Draw F1 0.041	Below the trees	Tested as the brief suggests: about 0.01 below Random Forest, with more tuning and less interpretability.
Linear SVM (§26)	Maximum-margin straight boundary; gives scores, not probabilities, so a calibration step is added.	P5/P8: one-vs-rest straight boundaries separate the middle class (Draw) poorly even with class weights (Draw F1 0.065 before calibration), and calibration maps scores back to true frequencies, which removes the rest of the draw predictions.	0.403 · Draw F1 0.000 · log loss 0.970	Poor for this goal	A linear boundary is no better than logistic regression, and with calibrated probabilities it almost never predicts a draw (once in 2,802 matches).
RBF kernel SVM (§26)	Maximum-margin linear boundary in an implicit high-dimensional space (Gaussian kernel), which is curved in the original features.	P1: training time grows about with rows²: 2.6 s (5,000 rows), 9.6 s (10,000), 195 s (42,795). P8: probabilities need an extra calibration step (several more training runs).	0.494 on all rows (−0.001 vs Random Forest) · Draw F1 0.329 · no log loss	Tie, not adopted	It reaches the same ceiling, but took about 195 s to train against about 6 s for Random Forest on this machine, and it needs extra calibration runs to give probabilities.

Model	How it works (theory and assumption)	Fit with our data (P1-P8)	Validation result	Verdict	If the evaluator asks "why not this model?"
Ordinal logistic regression (§26)	One latent "home advantage" score cut by two thresholds: A < D < H (proportional odds).	Matches the order in P5, but Draw is a narrow middle band: with the fitted thresholds P(Draw) can never exceed 0.29, so Draw is never the most probable class. (Unweighted multinomial logistic regression also predicts no draws, so class imbalance plus a weak draw signal is the main cause.)	Most probable class: 0.402, Draw F1 0.000. Prior-corrected: 0.475, Draw F1 0.271. Log loss 0.967 (good probabilities)	Useful insight, not adopted	It gives good probabilities, but needs a decision correction to predict draws and still ends 0.02 below Random Forest.
Poisson goal model (§27, script 06)	The classic statistical model for football: each team's goals are a Poisson count with log $\lambda = \beta_0 + \beta^T x$ from the same 38 features; two independent counts give the chance of every scoreline, which is summed into P(H), P(D) and P(A).	P2: the smooth signal suits a linear score, so its probabilities are good (P8). P5: two independent goal counts spread the chance of a draw thinly (the highest P(Draw) for any validation match is 0.29), so Draw is never the most probable result.	Most probable result: 0.405, Draw F1 0.000. Prior-corrected: 0.453, Draw F1 0.169. Log loss 0.969 (Random Forest 0.492 and 0.983 in the same run)	Useful insight, not adopted	Slightly better probabilities, but it cannot recognise draws: 0.039 below Random Forest (95% CI −0.054 to −0.024). The team-rating version (Dixon-Coles) was not run: it needs a continuous team history, which the 2016 source change breaks.
Deep learning (CNN, RNN, Transformer) (not run)	Learn representations from raw sequences, images or text.	P1: our input is a 38-column table, not raw sequences; the MLP result already shows no gain from neural capacity.	-	Not suitable	There is no raw sequential or image data for these architectures to exploit.
Stacking or voting ensembles (not run)	Combine the predictions of several models.	P6: the best models reach the same score from the same features; combining them cannot add information.	-	Low value	The best four models in Section 26 differ by at most 0.002; combining near-identical predictions adds complexity, not signal.

Colour of the model name: blue = selected · green = fits, effectively tied or useful · amber = works but weaker, or out of scope · red = assumption broken or poor for this goal.

3. Why do the best models tie?

- **Three different families reach the same level.** Bagging (Random Forest 0.495), boosting (HistGradientBoosting 0.495, XGBoost 0.493) and a kernel SVM (0.494) all land within 0.002 of each other in the same run, and the neural network comes close (0.485). When very different algorithms reach the same level, the limit is the information available before kickoff, not the algorithm.
- **The signal is mostly simple.** Plain logistic regression reaches 0.485 in the original run and 0.483 in the Section 26 run, about 0.01 below the best. Extra flexibility (deep trees, hidden layers, kernels) has little left to discover.
- **Football has a large random part.** The stronger XI fails to win about 3 in 10 matches that are not drawn. No model can predict that part.
- **What really separates models is the Draw class and probability quality.** Without class weighting, or a decision corrected for class shares, the linear, tree, boosting and neural models almost never predict a draw (Draw F1 0.00-0.04). Naive Bayes (log loss 2.35) and kNN (1.096, worse than the Dummy) give unreliable probabilities. Log loss is compared among class-weighted models, because weighting deliberately raises draw probabilities.
- **So the tie is broken by practical criteria.** Random Forest has joint-best Macro-F1, log loss within about 0.004 of the best class-weighted model, gives probabilities directly, trains in about 6 seconds, needs few settings and is explained through grouped feature importance. It was chosen before the extra models were run, and none of them beat it by the 0.01 margin.

4. Short answers

- **"Why not XGBoost or LightGBM?"** XGBoost and HistGradientBoosting (scikit-learn's LightGBM-style method) were tested: 0.493 and 0.495, against 0.495 for Random Forest in the same run. They tie; boosting cannot add information that is not in the features. AdaBoost, the original boosting method, scored lower (0.482), and Gradient Boosting's extra rounds raised only the training score (0.496 to 0.540 from 100 to 300 rounds) while validation stayed at 0.495-0.499.

- **"Why not a neural network?"** Tested (0.485): about 0.01 below Random Forest. Our data is a 38-feature table with a smooth main signal, where tree ensembles are usually at least as good as neural networks.
- **"The result is ordered (A < D < H). Why not ordinal regression?"** Tested. It gives good probabilities (log loss 0.967), but with the fitted thresholds P(Draw) never exceeds 0.29, so Draw is never the most probable outcome (Draw F1 0.00). With a prior correction it reaches 0.475, still below Random Forest.
- **"Why not SVM?"** A kernel SVM ties (0.494) but took about 195 seconds to train against about 6 for Random Forest, and it needs extra calibration runs to give the probabilities the stakeholder needs.
- **"Why not a statistical model, such as the Poisson goal model?"** Tested (notebook Section 27). It predicts each team's goals, the classic way to model football. Its probabilities are slightly better (log loss 0.969 against 0.983), but it never makes a draw the most likely result, so Macro-F1 is 0.405, or 0.453 with a class-share correction, against 0.492 for Random Forest.
- **"Why not the model with the best accuracy?"** The highest accuracies come from models that ignore draws (for example the unweighted MLP: 53.4% accuracy, Draw F1 0.04). Macro-F1 is the agreed metric because each outcome should count.

Evidence files: ml_outputs/candidate_ranking_validation.csv and tuned_validation_results.csv (original six); ml_outputs/additional_models_validation.csv (Section 26); ml_outputs/poisson_validation.csv (Section 27); R script 03 and 04 results (data properties).